Домашнее задание 1
Реализация синтаксического анализатора

В рамках домашнего задания необходимо реализовать синтаксический анализатор учебного языка Chocopy на языке C++. При реализации предлагается использовать подготовленный эскиз компилятора, содержащий реализацию лексического анализа, абстрактного синтаксического дерева и вспомогательных компонент для создания узлов дерева, и его вывода в JSON-формате. Кроме того, скелет содержит пример реализации синтаксического и семантического анализаторов, а также генератора LLVM-кода для арифметических выражений языка Chocopy. Синтаксический анализатор необходимо реализовать методом рекурсивного спуска без использования инструментов автоматического построения лексического и синтаксического анализаторов по формальному описанию грамматики языка.

СОДЕРЖАНИЕ:

1. Цели задания 2

1.1. Ожидаемый вывод 2

2. Структура скелета компилятора 2

3. Задачи домашнего задания 4

4. Система оценивания 4

1. Цели задания

Цель этого задания заключается в создании компилятора переднего плана (Frontend компилятора), который анализирует входную программу на языке Chocopy (проверяет принадлежность входной цепочки символов языку Chocopy) и создает абстрактное синтаксическое дерево (Abstract Syntax Tree — AST), которое может быть выведено в JSON-формате.

Запуск синтаксического анализатора и вывода построенного AST в JSON-формате выполняется следующей командой:

```
$ chocopy-llvm --ast-dump <INPUT>
```

где `<INPUT>` — путь до входного файла Chocopy.

1.1. Ожидаемый вывод

Синтаксический анализатор должен построить AST, структура которого соответствует приведенным в тестах проекта деревьям в JSON-формате.

Особое внимание следует уделить полю `location`, содержащему информацию о позиции языковой конструкции в файле с исходным кодом программы.

Для автоматизации проверки корректности синтаксического анализатора в скелете содержатся заранее подготовленные тесты. Эти тесты разбиты на 3 категории: `bad`, `stage1` и `stage2`. Категории `stage1`, `stage2` содержат тесты для проверки корректности AST для основных конструкций языка различных уровней сложности. Категория bad предназначена для проверки корректности обработки синтаксических ошибок, а также восстановления после ошибок в синтаксическом анализе. При обнаружении синтаксической ошибки необходимо восстановить синтаксический анализ в «`режиме паники`». Режим паники подразумевает пропуск токенов из входного потока до тех пор, пока не будет обнаружен токен из предопределенного множества синхронизирующих токенов. Например, пропуск токенов, принадлежащих текущей «логической строке» (см. спецификацию языка).

2. Структура скелета компилятора

В разделе предлагаются некоторые рекомендации к реализации синтаксического анализатора и описание основных инфраструктурных возможностей, которые уже реализованы в скелете.

На верхнем уровне проект организован следующим образом:

`include` — директория, содержащая заголовочные файлы проекта;

`include/chocopy-llvm/AST` — директория, содержащая заголовочные файлы с описанием узлов AST и объявлениями интерфейсов работы с ним;

`include/chocopy-llvm/Lexer` — директория, содержащая заголовочные файлы с объявлением интерфейсов лексического анализатора;

`include/chocopy-llvm/Parser` — директория, содержащая заголовочные файлы с объявлением интерфейсов синтаксического анализатора;

`include/chocopy-llvm/Sema` — директория, содержащая заголовочные файлы с объявлением интерфейсов семантического анализатора;

`lib` — директория, содержащая реализацию интерфейсов компонент компилятора;

`test/parser` — директория, содержащая набор тестов для проверки корректности синтаксического анализатора;

`test/demo` — директория, содержащая пример программы на языке Chocopy, поддерживаемой скелетом компилятора.

Предлагаемый эскиз компилятора содержит реализацию AST, интерфейсы для создания узлов дерева и вывода дерева в JSON-формате. Описание узлов AST расположено в соответствующем заголовочном файле:

`include/chocopy-llvm/AST/AST.h`

а для их создания используется `ASTContext`:

`include/chocopy-llvm/AST/ASTContext.h`.

Обратите внимание, для выделения памяти под узлы AST используется `BumpPtrAllocator`, реализация которого содержится в инфраструктуре llvm-project.

Вывод построенного AST в JSON-формате выполняется средствами специального класса `JSONDumper`. Описание этого класса содержится в заголовочном файле `include/chocopy-llvm/AST/JSONASTDumper.h`.

Для выполнения текущего домашнего задания необходимо реализовать синтаксический анализатор в файле `lib/Parser/Parser.cpp`.

3. Задачи домашнего задания

1) Изучить структуру проекта chocopy-llvm.

2) В классе `chocopy::Parser` реализовать синтаксический анализатор языка Chocopy методом рекурсивного спуска (в файле `lib/Parser/Parser.cpp`). При реализации необходимо руководствоваться грамматикой, приведенной в спецификации языка.

3) Добиться успешного прохождения тестов (`test/parser`).

4) Расширить существующий набор тестов новыми тестами категории `bad` и `stage[1-2]` (авторские тесты), содержащими языковые конструкции языка Chocopy, которые поддерживаются вашей реализацией синтаксического анализатора, но не покрыты имеющимися тестами. Авторские тесты необходимо разместить по следующему пути: `test/parser/students/<team>/<test>`, где `<team>` — название команды, а `<test>` — реализация теста (2 файла с расширением `.py` и `.py.ast`).

4. Система оценивания

Начисление баллов за выполнение домашнего задания будет выполняться следующим образом:

1) Реализация синтаксического анализатора для всей грамматики языка — **40 баллов**. Проверка осуществляется на имеющихся тестах.
2) Наличие авторских тестов — **5 баллов**.
3) Авторы лучших синтаксических анализаторов и авторских тестов, то есть авторы тестов, которые содержат программы, принадлежащие языку, задаваемому грамматикой (см. спецификацию Chocopy), но которые не разбираются анализаторами других команд, получают дополнительные **15 баллов**.

Система штрафов:

1) Каждый неподдержанный тест из категории `bad` или `stage[1-2]` оценивается в **-5 баллов**
2) Каждый поддержанный тест, но с ошибочной позицией в исходном файле (поле location в *.ast файле) оценивается в **-2 балла**.

Система вознаграждений:

1) Найденные ошибки в скелете компилятора и в тестах оцениваются в индивидуальном порядке.